DATABASE MANAGEMENT LAB

LAB Report

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF TECHNOLOGY

to

K.R Mangalam University

Submitted by:

Harsh Bardhan

(2401010130)

Submitted to

Ms. Mansi Kajal

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

SESSION – 2025-2026

Lab Sheet: 1

PART A – COLLEGE DATABASE

STUDENT (USN, SName, Address, Phone, Gender)
SEMSEC (SSID, Sem, Sec)
CLASS (USN, SSID)
SUBJECT (Subcode, Title, Sem, Credits)
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

Solution:

1. The entities **STUDENT, SEMSEC, SUBJECT, and IAMARKS** are identified and represented using rectangles.

2. The attributes are represented using ellipses:

o **STUDENT:** USN (Primary Key), SName, Address, Phone, Gender USN is underlined as it is the Primary Key.
 Phone is shown as a single-valued attribute.

o **SEMSEC:** SSID (Primary Key), Sem, Sec SSID is underlined as the Primary Key.

o **SUBJECT:** Subcode (Primary Key), Title, Sem, Credits Subcode is underlined as the Primary Key.

o **IAMARKS:** USN, Subcode, SSID, Test1, Test2, Test3, FinalIA USN, Subcode, and SSID together form a composite key. Test1, Test2, Test3, and FinalIA are normal attributes.

3. Primary keys are represented by underlining the attribute names.

4. **CLASS** is drawn as a relationship (diamond shape) between **STUDENT** and **SEMSEC**.

5. The relationship between **STUDENT and SEMSEC** through CLASS is **M : 1**, meaning many students belong to one semester and section.

6. The **M : N relationship between STUDENT and SUBJECT** is resolved using the associative entity **IAMARKS**.

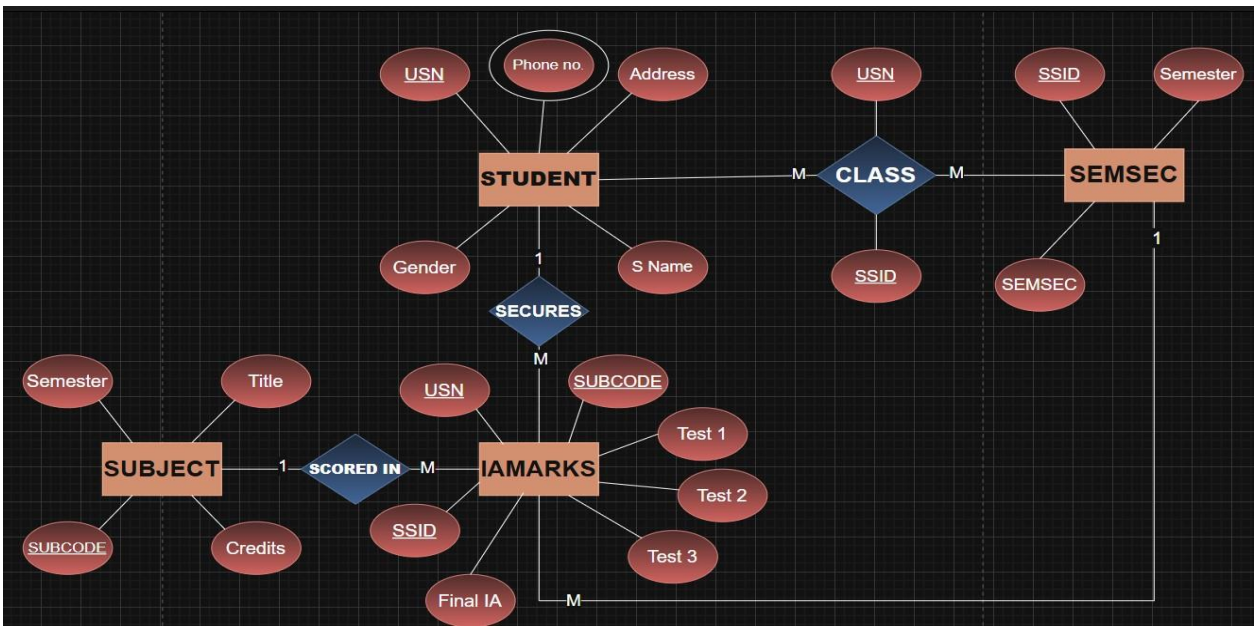
7. The relationship **STUDENT – IAMARKS** is **1 : M**, indicating one student can have multiple internal assessment records.

8. The relationship **SUBJECT – IAMARKS** is **1 : M**, indicating one subject can have multiple IA records for different students.

9. The relationship **SEMSEC – IAMARKS** is **1 : M**, showing that marks are maintained semestersection wise.

10. All entities, attributes, relationships, primary keys, foreign keys, and cardinalities are clearly represented in the ER diagram.

ER DIAGRAM:



PART B – COMPANY DATABASE

EMPLOYEE (SSN, Name, Address, Sex, Salary, Superssn, Dno)
DEPARTMENT (Dno, Dname, Mgrssn, Mgrstartdate)
DLOCATION (Dno, Dloc)
PROJECT (PNo, PName, PLocation, DNo)
WORKS_ON (SSN, PNo, Hours)

Solution:

1. The entities **EMPLOYEE, DEPARTMENT, PROJECT, and DLOCATION** are identified and represented using rectangles.

2. The attributes are represented using ellipses:

o **EMPLOYEE:** SSN (Primary Key), Name, Address, Salary, Sex, SuperSSN, DNO SSN is underlined as it is the Primary Key.
 SuperSSN is a foreign key referencing EMPLOYEE (self-relationship). DNO is a foreign key referencing DEPARTMENT.

o **DEPARTMENT:** DNo (Primary Key), DName, MgrSSN, MgrStartDate DNo is underlined as the Primary Key.

 MgrSSN is a foreign key referencing EMPLOYEE.

o **PROJECT:** PNo (Primary Key), PName, PLocation, DNO PNo is underlined as the Primary Key. DNO is a foreign key referencing DEPARTMENT.

o **DLOCATION:** DNo, DLoc

 DNo and DLoc together form a composite key. DNo is a foreign key referencing DEPARTMENT.

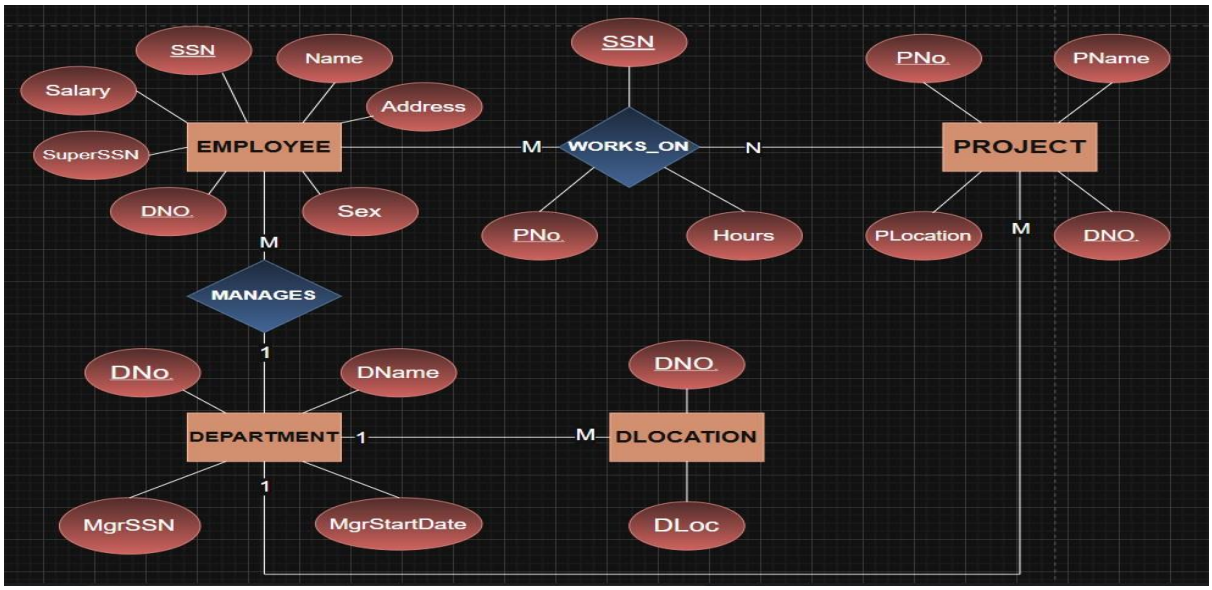
o **WORKS_ON:** SSN, PNo, Hours

 SSN and PNo together form a composite key.

 Hours is a descriptive attribute of the relationship.

3. Primary keys are represented by underlining the attribute names.
4. The relationship **WORKS_ON** is drawn as a diamond between **EMPLOYEE** and **PROJECT** with **M : N cardinality**, indicating that:
    - One employee can work on many projects. ○ One project can have many employees.
    - Hours attribute represents the number of hours worked.
5. The relationship **MANAGES** is drawn between **EMPLOYEE** and **DEPARTMENT** with **1 : 1 cardinality**, indicating:
    - One employee manages one department.
    - Each department has one manager.
6. The relationship between **EMPLOYEE and DEPARTMENT** (via DNO) is **M : 1**, meaning:
    - Many employees work in one department.
7. The relationship between **DEPARTMENT and PROJECT** is **1 : M**, meaning:
    - One department controls many projects. ○ Each project belongs to one department.
8. The relationship between **DEPARTMENT and DLOCATION** is **1 : M**, meaning: ○ One department can have multiple locations.
9. A recursive relationship exists in EMPLOYEE through **SuperSSN**, representing that:
    - One employee can supervise many employees.
10. All entities, attributes, relationships, primary keys, foreign keys, and cardinalities are clearly represented in the ER diagram.

ER DIAGRAM:



PART C – CUSTOMER SALE SCHEMA

Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name:string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

Solution:

Entities and Attributes

1. CUSTOMER Entity

The CUSTOMER entity stores information about customers who purchase products.

Attributes:

- Cust_ID – Unique identification number of a customer *(Primary Key)*
- Cust_Name – Name of the customer

Each customer is uniquely identified using Cust_ID.

2. SALE Entity

The SALE entity represents transaction or billing information whenever a customer buys items.

Attributes:

- Bill_No – Unique bill number *(Primary Key)*

- Bill_Date – Date on which purchase is made
- Qty_Sold – Quantity of items purchased

This entity acts as a bridge between customer and item details.

3. ITEM Entity

The ITEM entity contains details about products available for sale.

Attributes:

- Item_ID – Unique product identification number *(Primary Key)*
- Item_Name – Name of the item
- Price – Cost of the item

Each item is uniquely identified by Item_ID.

Relationships

1. MAKES Relationship

This relationship connects CUSTOMER and SALE.

- A Customer can make many sales.
- Each Sale belongs to only one customer.

Cardinality:

CUSTOMER (1) -------- MAKES -------- (M) SALE

Meaning:
One customer may generate multiple bills.

2. INCLUDES Relationship

This relationship connects SALE and ITEM.

- One Sale may include multiple items.
- Each Item can appear in many sales.

Cardinality:

SALE (M) -------- INCLUDES -------- (1) ITEM

Meaning:
A bill contains purchased items.

Key Concepts Used

- Entities represent real-world objects.
- Attributes describe entity properties.

- Primary Keys uniquely identify records.
- Relationships show interaction between entities.
- Cardinality defines how many entities participate in a relationship.

Relational Schema Derived

From the ER diagram, the following relational tables are obtained:

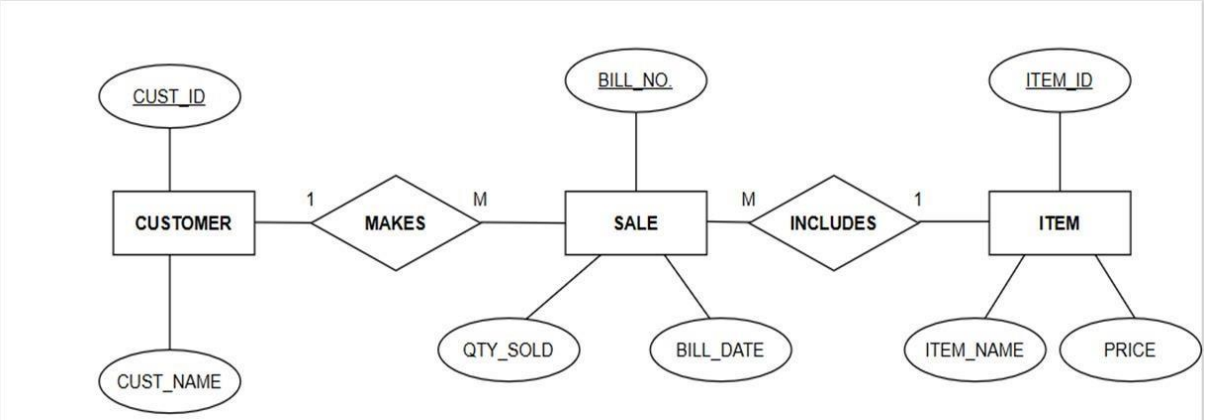
CUSTOMER (Cust_ID, Cust_Name)

ITEM (Item_ID, Item_Name, Price)

SALE (Bill_No, Bill_Date, Cust_ID, Item_ID, Qty_Sold) Where:

- Cust_ID is a Foreign Key referencing CUSTOMER.
- Item_ID is a Foreign Key referencing ITEM.

ER DIAGRAM:



DDL Questions (Data Definition Language):

1.) Write SQL statements to create all the above relations with appropriate PRIMARY KEY and FOREIGN KEY constraints.

STEP 1: CREATE DATABASE

```
CREATE DATABASE DBMS_Krish;
USE DBMS_Krish;
```

PART A – COLLEGE DATABASE :

1. CREATE TABLES

i) STUDENT TABLE :

```sql
CREATE TABLE STUDENT(
    USN VARCHAR(10) PRIMARY KEY,
    SName VARCHAR(50),
    Address VARCHAR(100),
    Phone VARCHAR(15),
    Gender CHAR(1)
);
```

ii) SEMSEC TABLE :

```sql
CREATE TABLE SEMSEC(
    SSID VARCHAR(10) PRIMARY KEY,
    Sem INT,
    Sec CHAR(1)
);
```

iii) CLASS TABLE:

```sql
CREATE TABLE CLASS(
    USN VARCHAR(10),
    SSID VARCHAR(10),
    PRIMARY KEY(USN, SSID),
    FOREIGN KEY(USN) REFERENCES STUDENT(USN),
    FOREIGN KEY(SSID) REFERENCES SEMSEC(SSID)
);
```

iv) SUBJECT TABLE :

```sql
CREATE TABLE SUBJECT(
    Subcode VARCHAR(10) PRIMARY KEY,
    Title VARCHAR(50),
    Sem INT,
    Credits INT
);
```

v) IMARKS TABLE :

```sql
CREATE TABLE IAMARKS(
    USN VARCHAR(10),
    Subcode VARCHAR(10),
    SSID VARCHAR(10),
    Test1 INT,
    Test2 INT,
    Test3 INT,
    FinalIA INT,
    PRIMARY KEY(USN,Subcode,SSID),
    FOREIGN KEY(USN) REFERENCES STUDENT(USN),
    FOREIGN KEY(Subcode) REFERENCES SUBJECT(Subcode),
    FOREIGN KEY(SSID) REFERENCES SEMSEC(SSID)
);
```

PART B – COMPANY DATABASE :

(i)DEPARTMENT TABLE :

```sql
CREATE TABLE DEPARTMENT(
    Dno INT PRIMARY KEY,
    Dname VARCHAR(50),
    Mgrssn INT,
    Mgrstartdate DATE
);
```

ii) EMPLOYEE TABLE :

```sql
CREATE TABLE EMPLOYEE(
    SSN INT PRIMARY KEY,
    Name VARCHAR(50),
    Address VARCHAR(100),
    Sex CHAR(1),
    Salary DECIMAL(10,2) NOT NULL,
    Superssn INT,
    Dno INT
);
```

iii) DLOCATION TABLE :

```sql
CREATE TABLE DLOCATION(
    Dno INT,
    Dloc VARCHAR(50),
    PRIMARY KEY(Dno,Dloc),
    FOREIGN KEY(Dno) REFERENCES DEPARTMENT(Dno)
);
```

iv) PROJECT TABLE :

```sql
CREATE TABLE PROJECT(
    PNo INT PRIMARY KEY,
    PName VARCHAR(50),
    PLocation VARCHAR(50),
    DNo INT,
    FOREIGN KEY(DNo) REFERENCES DEPARTMENT(Dno)
);
```

v) WORKS_ON TABLE :

```sql
CREATE TABLE WORKS_ON(
    SSN INT,
    PNo INT,
    Hours INT,
    PRIMARY KEY(SSN,PNo),
    FOREIGN KEY(SSN) REFERENCES EMPLOYEE(SSN),
    FOREIGN KEY(PNo) REFERENCES PROJECT(PNo)
);
```

PART C – CUSTOMER SALE SCHEMA:

i) CUSTOMER TABLE:

```
CREATE TABLE CUSTOMER(
    Cust_id INT PRIMARY KEY,
    cust_name VARCHAR(50)
);
```

ii) ITEM TABLE:

```
CREATE TABLE ITEM(
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50),
    price INT NOT NULL
);
```

iii) SALE TABLE :

```
CREATE TABLE SALE(
    bill_no INT,
    bill_date DATE,
    cust_id INT,
    item_id INT,
    qty_sold INT,
    PRIMARY KEY(bill_no,item_id),
    FOREIGN KEY(cust_id) REFERENCES CUSTOMER(Cust_id),
    FOREIGN KEY(item_id) REFERENCES ITEM(item_id)
);
```

2.) Add a new column Email to the STUDENT relation.

```
ALTER TABLE STUDENT
ADD Email VARCHAR(50);
SELECT * FROM STUDENT;
```

	USN	SName	Address	Phone	Gender	Email
*	NULL	NULL	NULL	NULL	NULL	NULL

3.)Apply NOT NULL constraints to Salary and Price attributes.

```
CREATE TABLE EMPLOYEE (
    SSN VARCHAR(20) PRIMARY KEY,
    Name VARCHAR(50),
    Address VARCHAR(100),
    Sex CHAR(1),
    Salary DECIMAL(10,2) NOT NULL,
    Superssn VARCHAR(20),
    Dno INT,
    FOREIGN KEY (Superssn) REFERENCES EMPLOYEE(SSN),
    FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dno)
);
```

IN EMPLOYEE TABLE SALARY IS NOT NULL .

```
CREATE TABLE ITEM (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50),
    price INT NOT NULL
);
```

4.)Rename attribute SName to StudentName in STUDENT relation.

```
ALTER TABLE STUDENT
RENAME COLUMN SName TO StudentName;
SELECT * FROM STUDENT;
```

	USN	StudentName	Address	Phone	Gender	Email
*	NULL	NULL	NULL	NULL	NULL	NULL

5.) Drop the DLOCATION relation.

```
DROP TABLE DLOCATION;
```

DML Questions (Data Manipulation Language):

Question : Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

Output:

```
CREATE DATABASE CUSTOMER_SALE;
USE CUSTOMER_SALE;
CREATE TABLE CUSTOMER(
CUST_ID INT PRIMARY KEY,
CUST_NAME VARCHAR(100)
);
CREATE TABLE ITEM (
ITEM_ID INT PRIMARY KEY,
ITEM_NAME VARCHAR(100),
PRICE INT);
CREATE TABLE SALE (
BILL_NO INT ,
BILL_DATA DATE,
CUST_ID INT,
ITEM_ID INT,
QTY_SOLD INT ,
FOREIGN KEY (CUST_ID) REFERENCES CUSTOMER(CUST_ID),
FOREIGN KEY (ITEM_ID) REFERENCES ITEM(ITEM_ID)
);
```

2. Insert around 10 records in each of the tables.

```sql
INSERT INTO
CUSTOMER(CUST_ID ,CUST_NAME)
VALUES
(1,'Krish'),
(2,'Harsh'),
(3,'Ankit'),
(4,'Aryan'),
(5,'Hridyanshu'),
(6,'Ashok'),
(7,'Alok'),
(8,'Aman'),
(9,'Ashish'),
(10,'Mohit');
```

CUST_ID	CUST_NAME
1	Krish
2	Harsh
3	Ankit
4	Aryan
5	Hridyanshu
6	Ashok
7	Alok
8	Aman
9	Ashish
10	Mohit
NULL	NULL

```sql
INSERT INTO ITEM VALUES
(101,'Pen',50),
(102,'Notebook',120),
(103,'Bag',800),
(104,'Bottle',250),
(105,'Shoes',1500),
(106,'Watch',2200),
(107,'Mouse',500),
(108,'Keyboard',700),
(109,'Charger',300),
(110,'Headphones',1800);
SELECT * FROM ITEM;
```

ITEM_ID	ITEM_NAME	PRICE
101	Pen	50
102	Notebook	120
103	Bag	800
104	Bottle	250
105	Shoes	1500
106	Watch	2200
107	Mouse	500
108	Keyboard	700
109	Charger	300
110	Headphones	1800
NULL	NULL	NULL

```sql
INSERT INTO SALE VALUES
(1,CURDATE(),1,103,1),
(2,CURDATE(),2,101,3),
(3,CURDATE(),2,105,2),
(4,'2026-02-15',3,104,1),
(5,'2026-02-14',4,106,1),
(6,'2026-02-13',5,102,4),
(7,CURDATE(),6,110,1),
(8,'2026-02-16',7,108,2),
(9,CURDATE(),5,109,5),
(10,'2026-02-12',8,107,1);
SELECT * FROM SALE;
```

BILL_NO	BILL_DATA	CUST_ID	ITEM_ID	QTY_SOLD
1	2026-02-23	1	103	1
2	2026-02-23	2	101	3
3	2026-02-23	2	105	2
4	2026-02-15	3	104	1
5	2026-02-14	4	106	1
6	2026-02-13	5	102	4
7	2026-02-23	6	110	1
8	2026-02-16	7	108	2
9	2026-02-23	5	109	5
10	2026-02-12	8	107	1

3.List all the bills for the current date with the customer names and item numbers

```
SELECT S.BILL_NO, C.CUST_NAME, S.ITEM_ID
FROM SALE S
JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID
WHERE S.BILL_DATA = CURDATE();
```

BILL_NO	CUST_NAME	ITEM_ID
1	Krish	103
2	Harsh	101
3	Harsh	105
7	Ashok	110
9	Hridyanshu	109

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT
    S.BILL_NO,
    C.CUST_NAME,
    I.ITEM_NAME,
    S.QTY_SOLD,
    I.PRICE,
    (S.QTY_SOLD * I.PRICE) AS FINAL_AMOUNT
FROM SALE S
JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;
```

BILL_NO	CUST_NAME	ITEM_NAME	QTY_SOLD	PRICE	FINAL_AMOUNT
1	Krish	Bag	1	800	800
2	Harsh	Pen	3	50	150
3	Harsh	Shoes	2	1500	3000
4	Ankit	Bottle	1	250	250
5	Aryan	Watch	1	2200	2200
6	Hridyanshu	Notebook	4	120	480
7	Ashok	Headphones	1	1800	1800
8	Alok	Keyboard	2	700	1400
9	Hridyanshu	Charger	5	300	1500
10	Aman	Mouse	1	500	500

5.List the details of the customer who have bought a product which has a price>200

```
SELECT DISTINCT C.*
FROM CUSTOMER C
JOIN SALE S ON C.CUST_ID = S.CUST_ID
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE I.PRICE > 200;
```

CUST_ID	CUST_NAME
1	Krish
3	Ankit
2	Harsh
4	Aryan
8	Aman
7	Alok
5	Hridyanshu
6	Ashok

6.Give a count of how many products have been bought by each customer

```
SELECT C.CUST_NAME, SUM(S.QTY_SOLD) AS TOTAL_PRODUCTS_BOUGHT
FROM CUSTOMER C
JOIN SALE S ON C.CUST_ID = S.CUST_ID
GROUP BY C.CUST_NAME;
```

CUST_NAME	TOTAL_PRODUCTS_BOUGHT
Krish	1
Harsh	5
Ankit	1
Aryan	1
Hridyanshu	9
Ashok	1
Alok	2
Aman	1

7.Give a list of products bought by a customer having cust_id as 5.

```
SELECT I.ITEM_NAME, S.QTY_SOLD
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE S.CUST_ID = 5;
```

ITEM_NAME	QTY_SOLD
Notebook	4
Charger	5

8.List the item details which are sold as of today.

```
SELECT I.*
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE S.BILL_DATA = CURDATE();
```

ITEM_ID	ITEM_NAME	PRICE
103	Bag	800
101	Pen	50
105	Shoes	1500
110	Headphones	1800
109	Charger	300

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold,Amount

```
CREATE VIEW Bill_Details AS
SELECT
    S.BILL_NO,
    S.BILL_DATA,
    S.CUST_ID,
    S.ITEM_ID,
    I.PRICE,
    S.QTY_SOLD,
    (S.QTY_SOLD * I.PRICE) AS AMOUNT
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;

SELECT * FROM BILL_DETAILS;
```

BILL_NO	BILL_DATA	CUST_ID	ITEM_ID	PRICE	QTY_SOLD	AMOUNT
1	2026-02-23	1	103	800	1	800
2	2026-02-23	2	101	50	3	150
3	2026-02-23	2	105	1500	2	3000
4	2026-02-15	3	104	250	1	250
5	2026-02-14	4	106	2200	1	2200
6	2026-02-13	5	102	120	4	480
7	2026-02-23	6	110	1800	1	1800
8	2026-02-16	7	108	700	2	1400
9	2026-02-23	5	109	300	5	1500
10	2026-02-12	8	107	500	1	500

10.Create a view which lists the daily sales date wise for the last one week

```sql
CREATE VIEW WEEKLY_SALES AS
SELECT
    BILL_DATA,
    SUM(S.QTY_SOLD * I.PRICE) AS TOTAL_DAILY_SALES
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE BILL_DATA >= CURDATE() - INTERVAL 7 DAY
GROUP BY BILL_DATA;

SELECT * FROM WEEKLY_SALES;
```

BILL_DATA	TOTAL_DAILY_SALES
2026-02-23	7250
2026-02-16	1400